Network Evolution Analysis

Team Name: Group 16
Members:

- Vatsal Bhuva (IIT2022004)
- Aayush Aeron (IIT2022112)
- Kotesh Boyina (IIT2022235)

Introduction

This project presents a privacy-aware system to analyze the evolution of hoax call networks using **dynamic graph analysis** and **machine learning techniques**. Leveraging open-source libraries and public datasets, the system models temporal communication networks and detects structural changes, community patterns, behavioral anomalies, and evolving fraud language. The framework integrates time-series change detection, growth model simulation, semantic embeddings, and interactive visual analytics to support threat intelligence in fraudulent communication environments.

Objective

To build a privacy-aware system that uses **temporal network modeling**, **natural language processing (NLP)**, and **social network analysis (SNA)** to detect structural changes, emerging influencers, and suspicious language trends in hoax call networks. The system also simulates network growth and enables dynamic visualization to support investigation.

Dataset(s) Used

- **Source:** Kaggle — Fraud Call Detection Dataset
- **Content:** Call transcripts labeled as `"fraud"` or `"normal"`

- **Preprocessing included:**
    - Removal of null/malformed entries
    - Standardizing to comma-separated text format
    - Tokenization, stopword filtering
    - Year-wise segmentation using metadata

Literature Review:

The field of dynamic graph analysis has seen significant growth, driven by the need to model evolving real-world networks, such as social, financial, and communication systems. Various methods have been proposed to embed temporal information, handle structural evolution, and ensure efficient learning on time-varying graph data. The following review explores major contributions in this area.

1. **Temporal Network Embedding Approaches**

Chen et al. (2019) propose a dynamic network embedding method leveraging random walks and Bernoulli embeddings to capture temporal continuity without requiring explicit alignment steps [1]. This model effectively maintains node proximity and reflects network evolution across time snapshots, evaluated via link prediction and change detection.

In contrast, Qi et al. (2020) focus on modeling triadic closure processes through their DynamicTriad approach [7], simulating the likelihood of open triads becoming closed. This mechanism not only captures structural relationships but also drives the evolution of node representations.

2. **Self-Attention and Recurrent Architectures**

A breakthrough in attention-based learning on graphs, DySAT by Sankar et al. [3] and Huang et al. [9], applies self-attention across both structural and temporal dimensions. This approach enhances dynamic node representation by weighing dependencies adaptively over time, outperforming baselines in tasks like dynamic link prediction.

Similarly, EvolveGCN by El-Kishky et al. [4] extends traditional GCNs to dynamic settings by updating their parameters through Recurrent Neural Networks (RNNs). This eliminates the need for static embeddings and better accommodates evolving node and edge structures.

3. **Continuous-Time Dynamic Modeling**

Moving beyond discrete time, Hamilton et al. introduce a framework based on Temporal Dependency Interaction Graphs (TDIG) [5], enabling learning over continuous-time dynamic graphs. This approach models asynchronous interaction events and facilitates fine-grained temporal reasoning for forecasting network evolution.

Rossi et al. (2020) further develop this idea with Temporal Graph Networks (TGN) [8], which combine static features and dynamic interactions into a unified inductive framework. TGN supports scalability and adaptability for real-world systems where node arrivals and temporal variance are unpredictable.

4. **Network Structure and Content Coevolution**

Teng et al. [2] explore a less traditional yet critical aspect: the coevolution of network structure and information content. Their findings suggest a direct relationship between structural conductance and information entropy, indicating that network topology can be predictive of communication richness and novelty, especially in online ecosystems.

Rossi et al. (2020) introduce Temporal Graph Networks (TGN), a highly flexible and scalable framework for inductive representation learning on dynamic graphs. Unlike traditional models that require static snapshots or retraining for new data, TGN handles continuous-time interaction data and can generalize to unseen nodes and edges.

5. **Surveys and Taxonomy of Methods**

Goyal et al. [6] present a comprehensive survey on representation learning in dynamic graphs, classifying models using an encoder-decoder paradigm. The paper discusses the landscape of inductive and transductive learning, highlights benchmark datasets, and points out open challenges, such as scalability and handling missing data.

Additionally, Yu et al. [10] survey Spatio-Temporal Graph Neural Networks (STGNNs), which incorporate both spatial and temporal dependencies. These networks are particularly suited for domains like traffic forecasting and human activity recognition, where spatial structure and temporal progression are equally important.

References

1. Chen, C., Tao, Y., Lin, H. (2019). *Dynamic Network Embeddings for Network Evolution Analysis*. arXiv.

2. Teng, C.-Y., Gong, L., Livne, A., Brunetti, C., Adamic, L. A. *Coevolution of Network Structure and Content*. ADS.

3. Sankar, A., Dong, Y., Leskovec, J. *Dynamic Node Embeddings for Evolving Graphs*.

4. El-Kishky, M., et al. (2019). *EvolveGCN: Evolving Graph Convolutional Networks for Dynamic Graphs*. arXiv.

5. Hamilton, W. L., Ying, R., Leskovec, J. *Continuous-Time Dynamic Graph Learning via Neural Interaction Processes*. ACM DL.

6. Goyal, P., et al. (2019). *Representation Learning for Dynamic Graphs: A Survey*. arXiv.

7. Qi, F., Wang, X., Zhu, W. (2020). *Dynamic Network Embedding by Modeling Triadic Closure Process*. AAAI.

8. Rossi, E., Chamberlain, B., Frasca, F., Monti, F. (2020). *Inductive Representation Learning on Temporal Graphs*. arXiv.

9. Huang, W., et al. *Dynamic Graph Representation Learning via Self-Attention Networks*. arXiv/OpenReview.

10. Yu, B., Yin, H., Zhu, Z. (2023). *Spatio-Temporal Graph Neural Networks: A Survey*. arXiv.

.

Methodology:

1. Preprocessing

- Loaded the hoax call dataset and removed null, duplicate, or malformed entries.
- Reformatted raw messages into **comma-separated** structured format suitable for analysis.
- Applied **tokenization**, **lowercasing**, and **stopword removal** using `NLTK` to extract clean, meaningful words from each message.

2. Entity Extraction

- Treated each **unique word** in the cleaned transcripts as a **node** in the network.
- This allowed modeling the linguistic structure as a word interaction graph.
- The tokenized messages were also reused in **Word2Vec** training and sentence reconstructions for embedding generation.

3. Edge Construction

- Created **edges** between every pair of words that co-occurred in the same message.
- Used a **co-occurrence model** where each message is a clique, connecting all words that appeared together.
- Edge weights reflect the **frequency** of co-occurrence across the dataset.

4. Temporal Modeling

- Leveraged **year-wise segmentation** to analyze how the word network evolved over time.
- Used `DyNetX`, a dynamic graph extension of NetworkX, to build temporal snapshots and observe structural shifts.
- Calculated and visualized network metrics like degree, density, clustering coefficient over years.

5. Embeddings

➤ **Sentence Embeddings (Semantic-Level)**

- Used `SentenceTransformer (all-MiniLM-L6-v2)` to convert each message into a **384-dimensional semantic vector**.
- Captures full-message meaning, which was used for:
  - **Clustering** (with KMeans and UMAP)
  - **Training a classifier**
  - **Visualizing fraud vs normal messages** via 2D projection

➤ **Word Embeddings (Context-Level)**

- Trained a `Word2Vec` model on the tokenized messages.
- Generated **100-dimensional vectors** for individual words based on their surrounding context.
- Used UMAP to project these vectors into 2D and visualize **fraud-associated vocabulary** by coloring words based on their dominant usage in either fraud or normal messages.

6. Classifier Training & Evaluation

- Trained a **Logistic Regression** model using sentence embeddings as input features.
- Applied `class_weight='balanced'` to address class imbalance (fewer fraud messages than normal).
- Evaluated performance using **precision, recall, F1-score**, and **confusion matrix**.
- Also analyzed **feature importance** by correlating model weights with embedding dimensions, identifying high-impact fraud-related terms (e.g., "otp", "account").

7. Growth Simulation

- Used the **Watts-Strogatz model** to simulate small-world network growth.
- Compared its properties (clustering, path length, modularity) with real network snapshots.
- Helped assess whether hoax call networks resemble naturally evolving communication patterns.

Libraries Used:

- `NetworkX` & `DyNetX`: Graph creation and dynamic graph handling

- `Ruptures`: Change point detection in network metric time series
- `SentenceTransformers` & `Gensim`: Sentence and word embeddings
- `Matplotlib`, `Seaborn`, `Plotly`, `Bokeh`: Visual analytics and dashboard
- `scikit-learn`: Classifier, clustering, evaluation metrics
- `UMAP`: Dimensionality reduction for embedding visualization

System Architecture/Workflow:

1. Load & Clean Dataset
2. Extract Words as Nodes
3. Build Word Co-occurrence Graphs
4. Segment Network Year-wise using DyNetX
5. Analyze Metrics & Detect Communities
6. Apply Change Point Detection (Ruptures)
7. Create sentence and word embeddings for semantic analysis
8. Train classification model on sentence embeddings
9. Visualize with NetworkX, Plotly, and Bokeh Dashboard

Key Features Implemented:

1. Community Detection via Greedy Modularity

- Applied greedy modularity optimization to identify **clusters of related words** within the word co-occurrence network.
- Helps in discovering **semantic communities**, which may reflect specific types of messaging patterns, such as fraud-related language.

2. Static and Dynamic Graph Visualization

- Constructed and visualized both **static** word co-occurrence graphs and **dynamic** (year-wise) snapshots using NetworkX and DyNetX.
- Enabled analysis of how **communication structures evolve** over time within the hoax call ecosystem.

3. Bokeh-based Interactive Dashboard

- Developed a **web-based interactive dashboard** using Bokeh for real-time exploration of graphs, metrics, and embeddings.

- Provided user-friendly controls to filter by year, label, and node importance for investigative purposes.

4. Change Detection in Edge/Node Patterns (Ruptures)

- Integrated the **Ruptures** library to perform **change point detection** on time series of graph metrics (e.g., edge count, density).

- Identified significant structural shifts, which can correlate with changes in fraud behavior or new scam campaigns.

5. Watts-Strogatz Growth Model Simulation

- Simulated a **small-world network** using the Watts-Strogatz model to compare against real hoax call networks.

- Assessed the **structural realism** of real data by comparing graph metrics like average path length and clustering coefficient.

6. Link Prediction using Common Neighbors & Precision-Recall Evaluation

- Implemented **temporal link prediction** to estimate the likelihood of new word associations appearing in future network snapshots.

- Used **Common Neighbors** as a scoring function and evaluated performance using **precision and recall** metrics.

7. Centrality and Graph Metric Tracking

- Computed key centrality measures including **degree**, **betweenness**, and **closeness** centrality across time.

- Tracked global graph metrics like **density**, **average clustering coefficient**, and **average path length** for network health analysis.

8. Sentence Embedding and Clustering

- Used **Sentence-BERT** to generate semantic embeddings of each call transcript.
- Applied **KMeans clustering** and **UMAP projection** to visualize and group similar messages, revealing clear separations between fraud and normal calls.

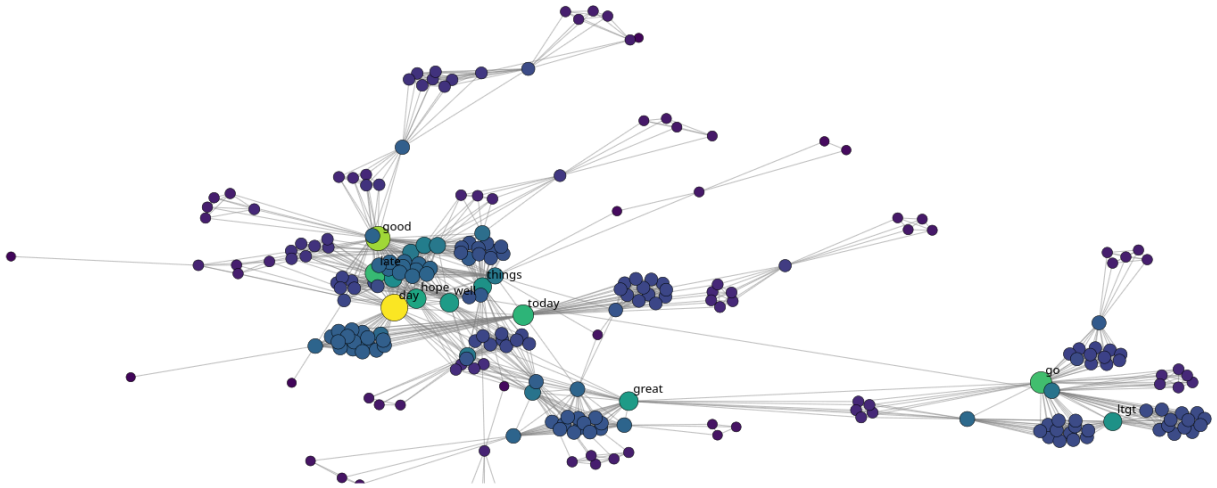
9. Fraud Classifier with Class Imbalance Handling

- Trained a **Logistic Regression** model using sentence embeddings to classify calls as fraud or normal.
- Handled dataset imbalance using `class_weight='balanced'` to ensure fair learning despite fewer fraud examples.
- Evaluated using standard classification metrics and saved predictions and model artifacts

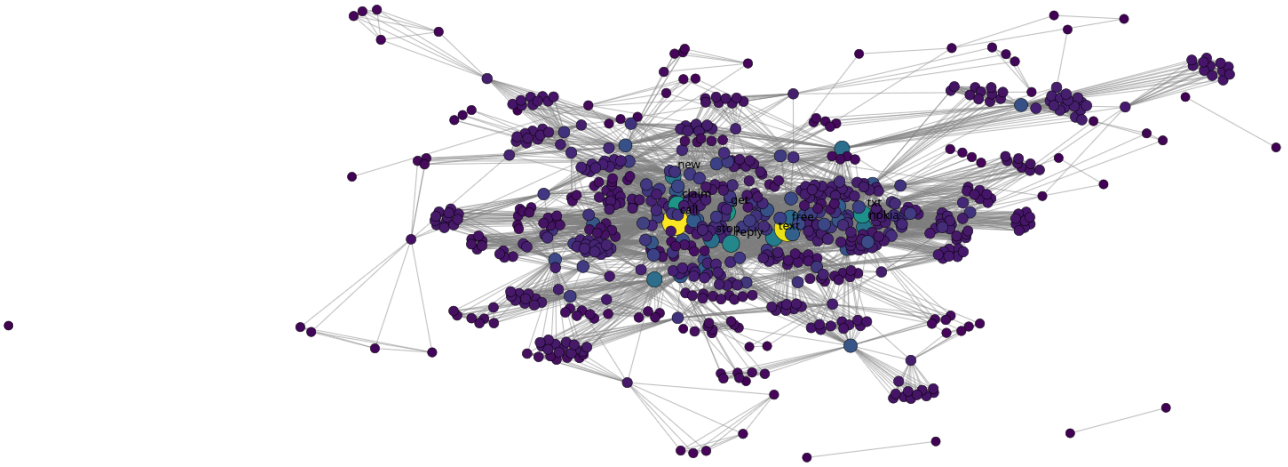
Visualizations & Results:

- Plots showing yearly word networks (DyNetX snapshots)

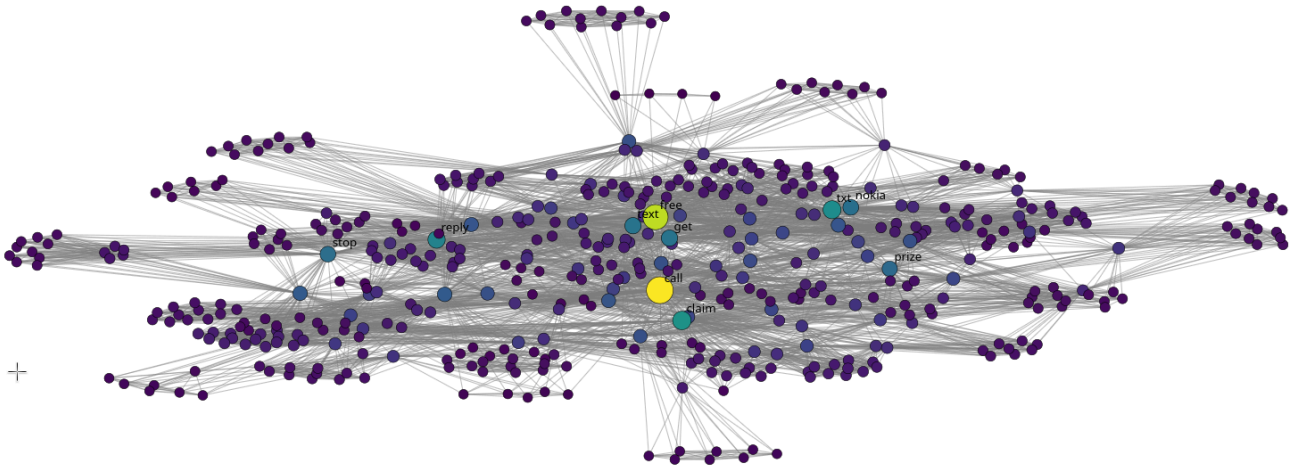
Normal Message Network for 2022



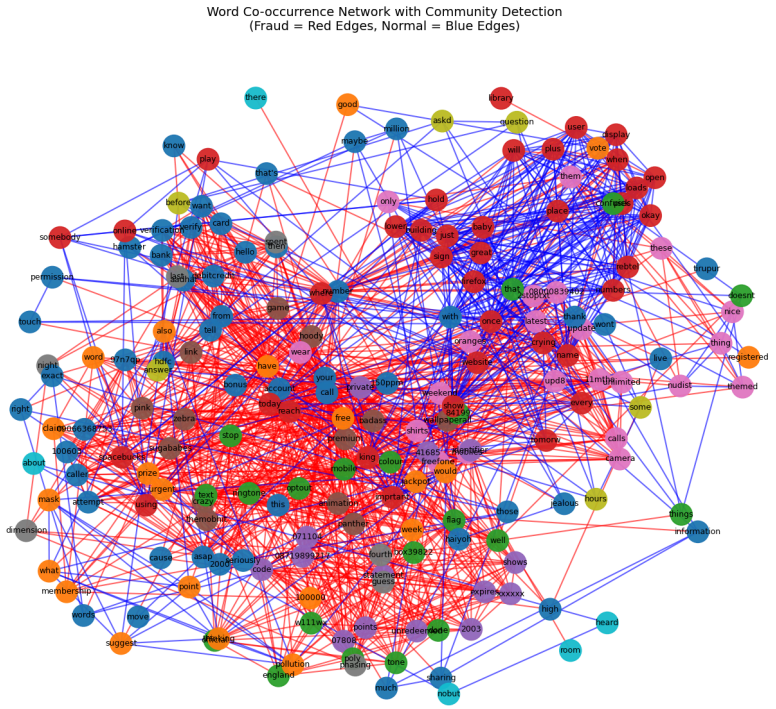
Word Co-occurrence Network for 2022

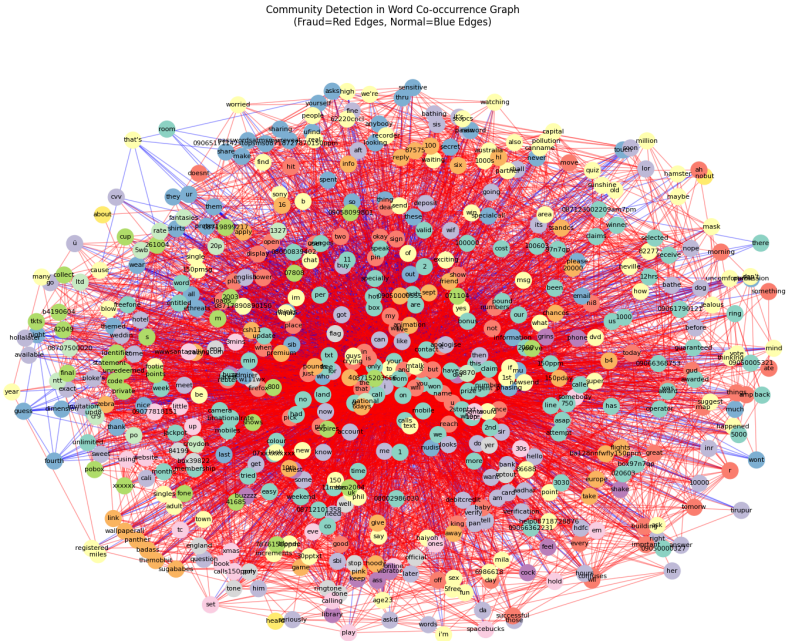


Fraud Message Network for 2022

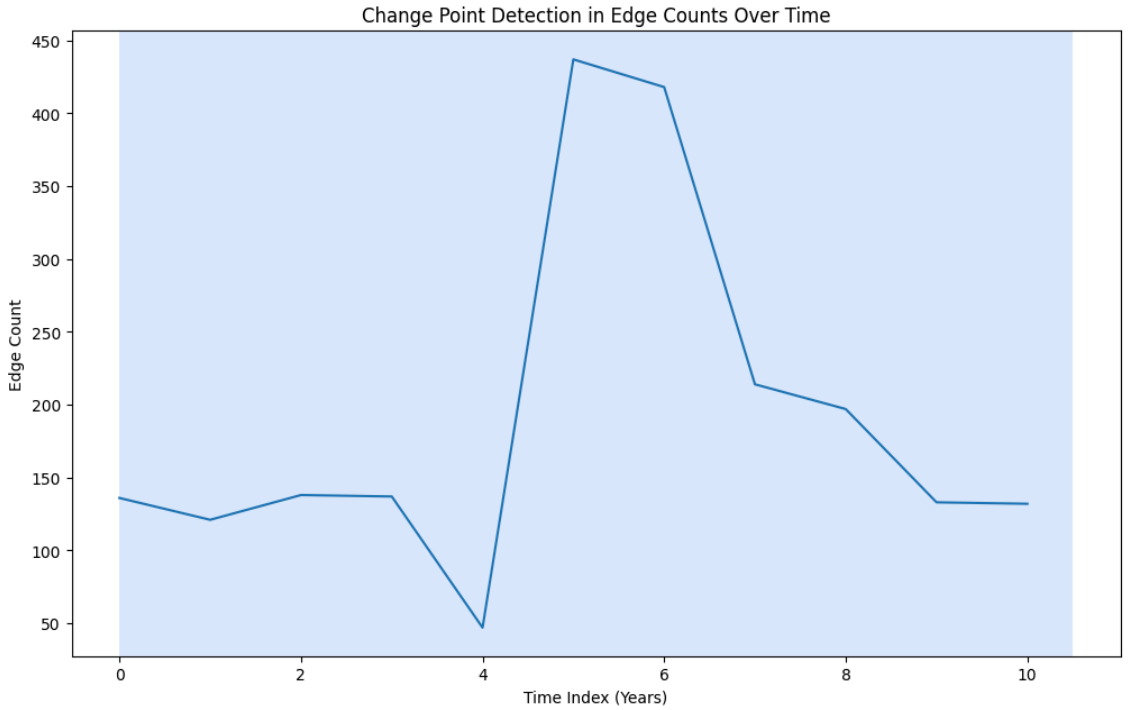


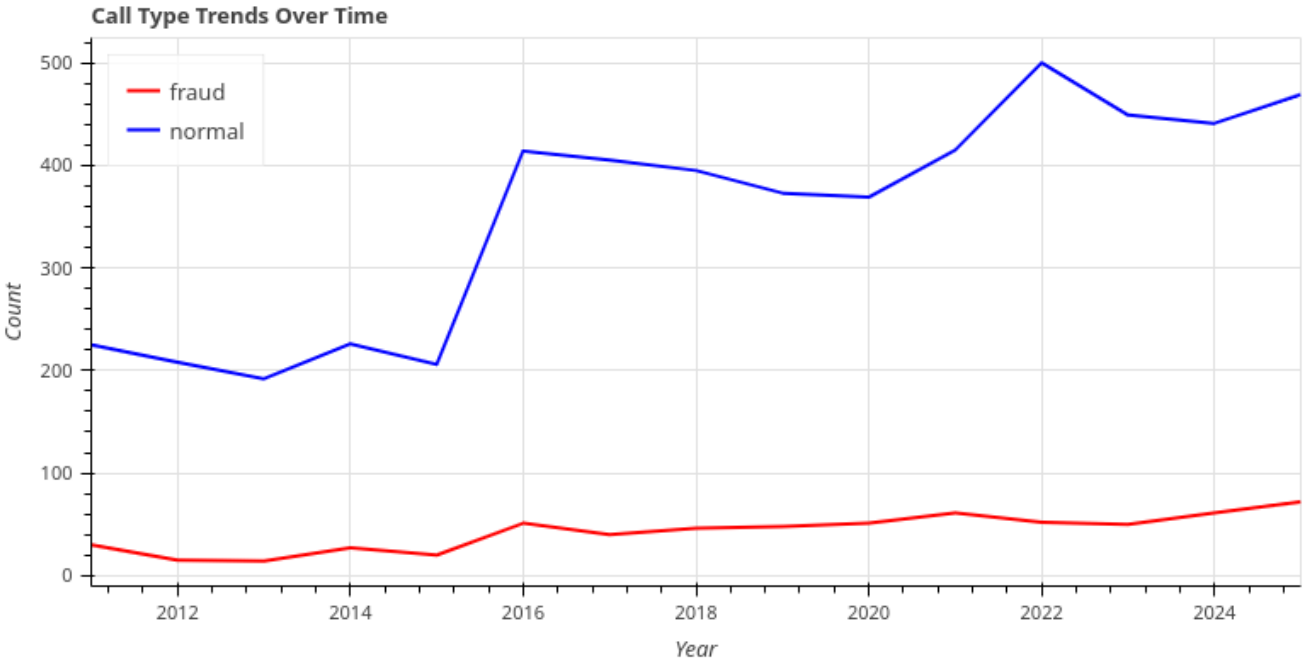
- Community detection (Color-coded graphs)



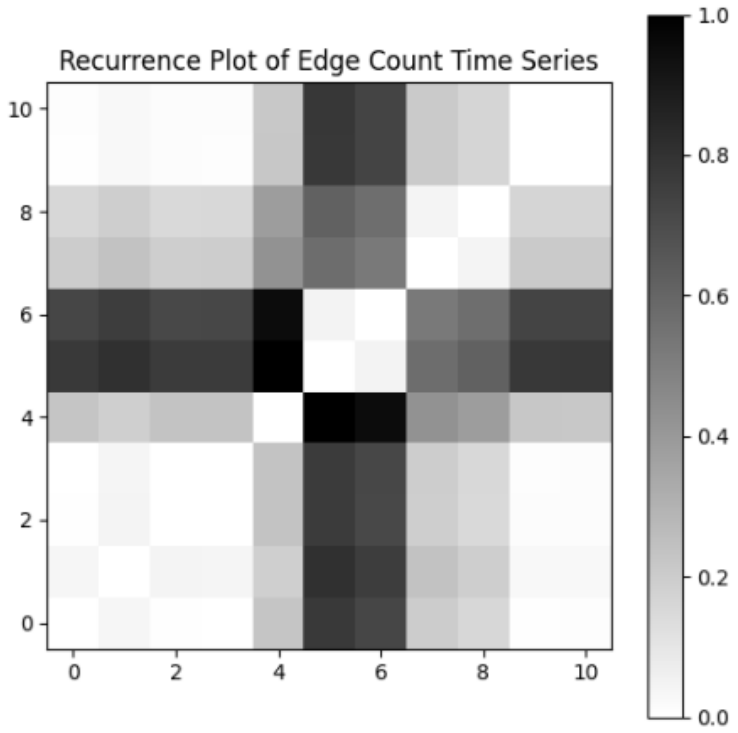


- Time-series of edge-counts and clustering coefficients

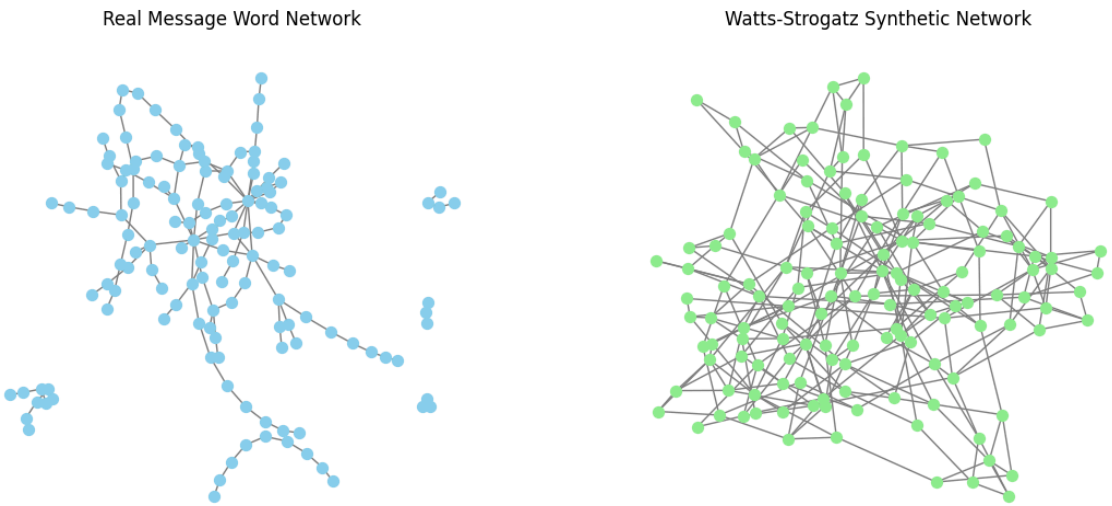


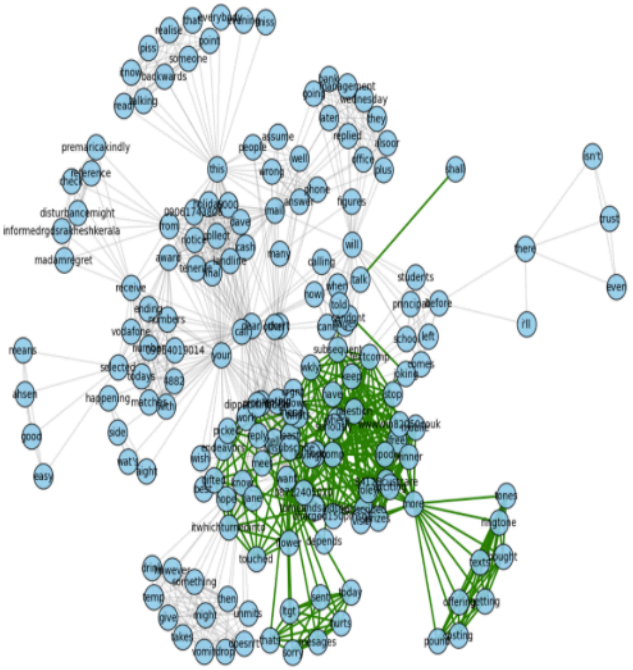


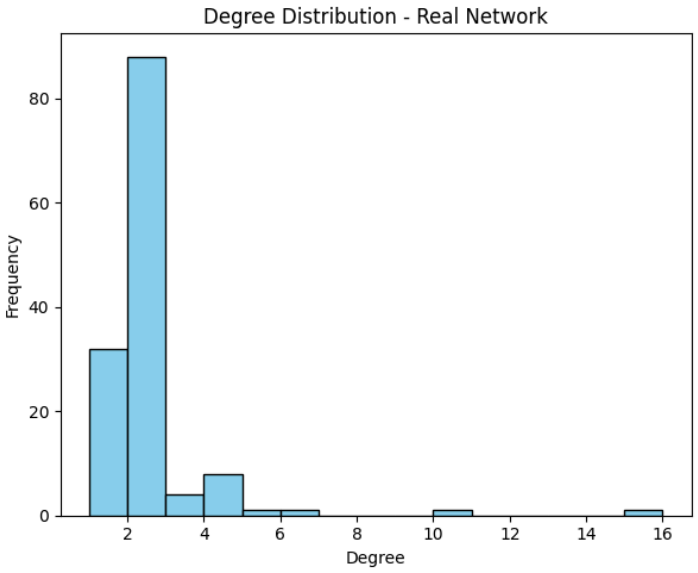
- Recurrence plot comparing network structure across years

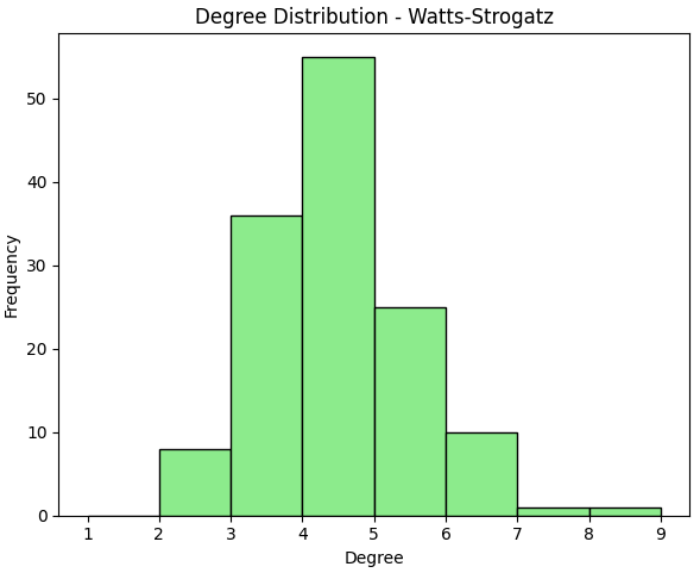


- Side-by-side comparisons of real vs WS Model graphs

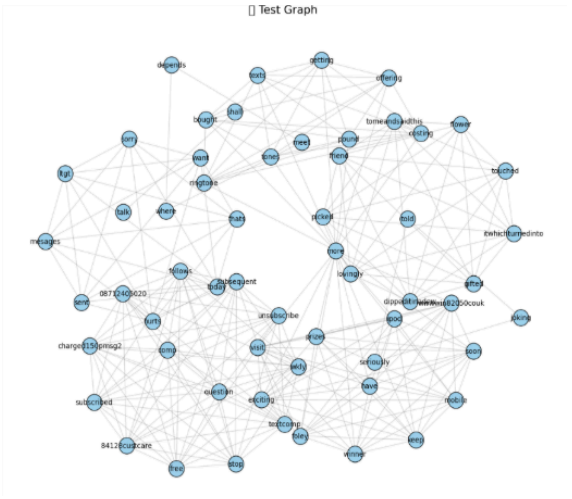


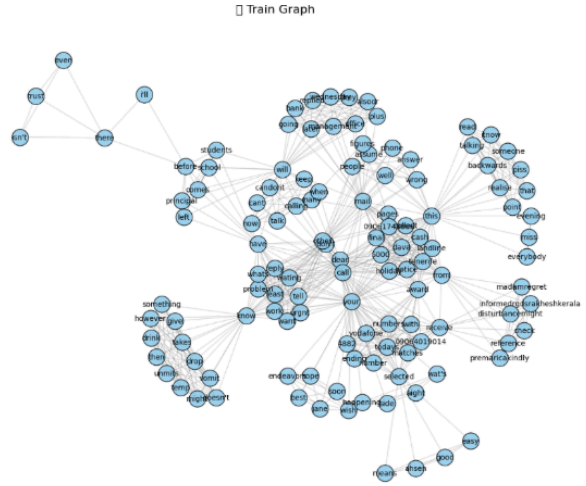


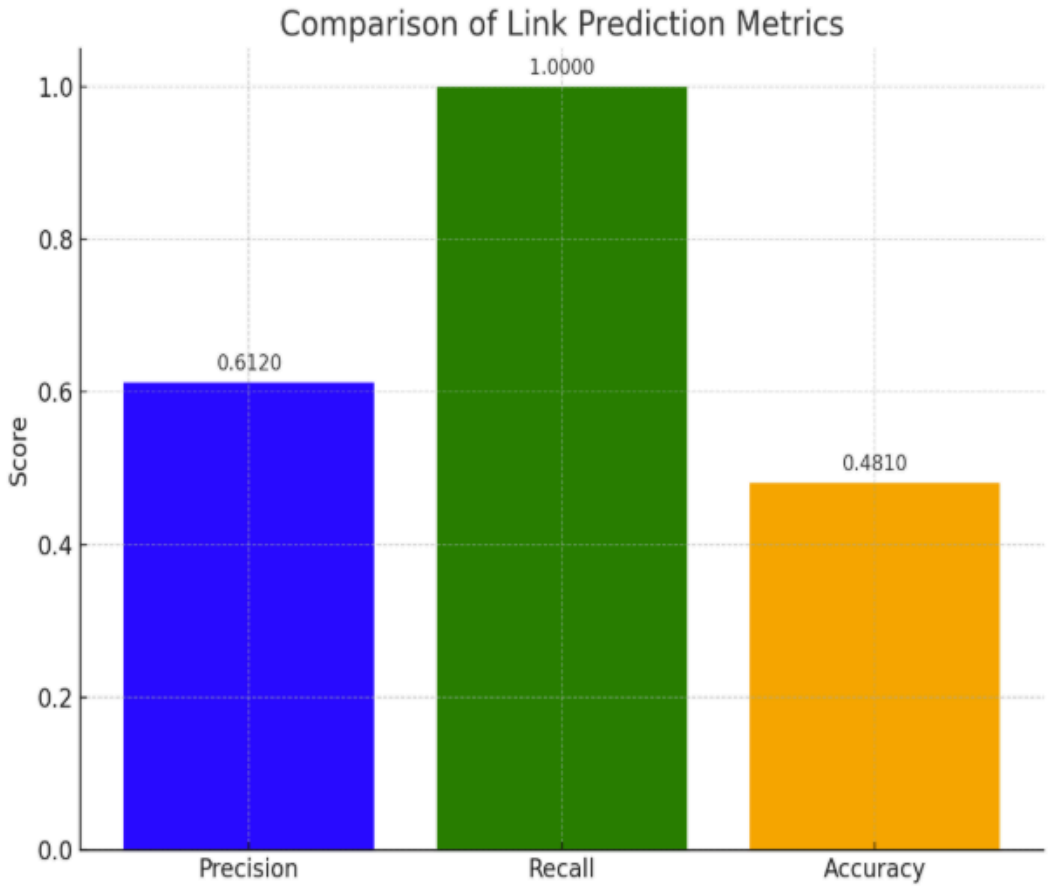

Degree Distribution - Real Network
Frequency
Degree
0
20
40
60
80
2
4
6
8
10
12
14
16

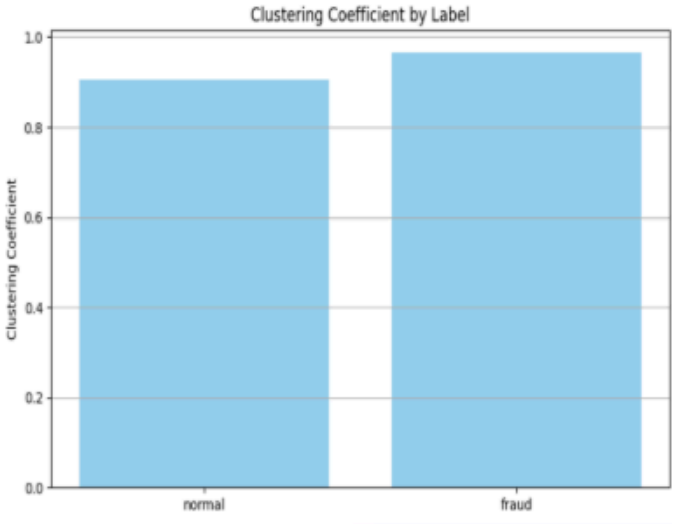

Degree Distribution - Watts-Strogatz
Frequency
Degree
0
10
20
30
40
50
1
2
3
4
5
6
7
8
9

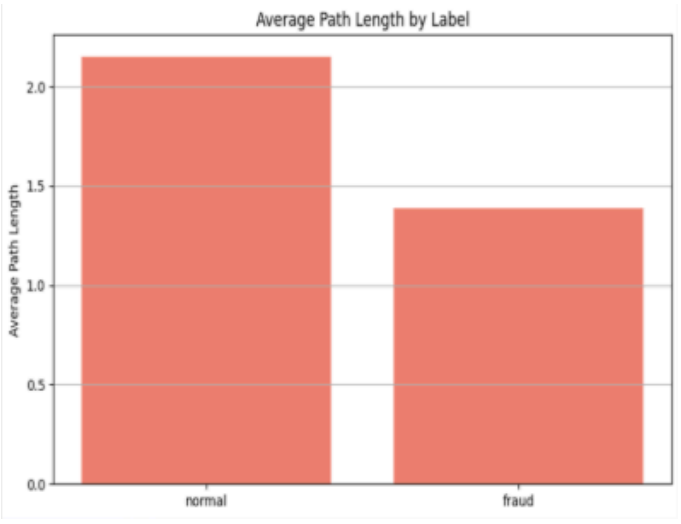
- Link prediction visualizations and accuracy evaluations



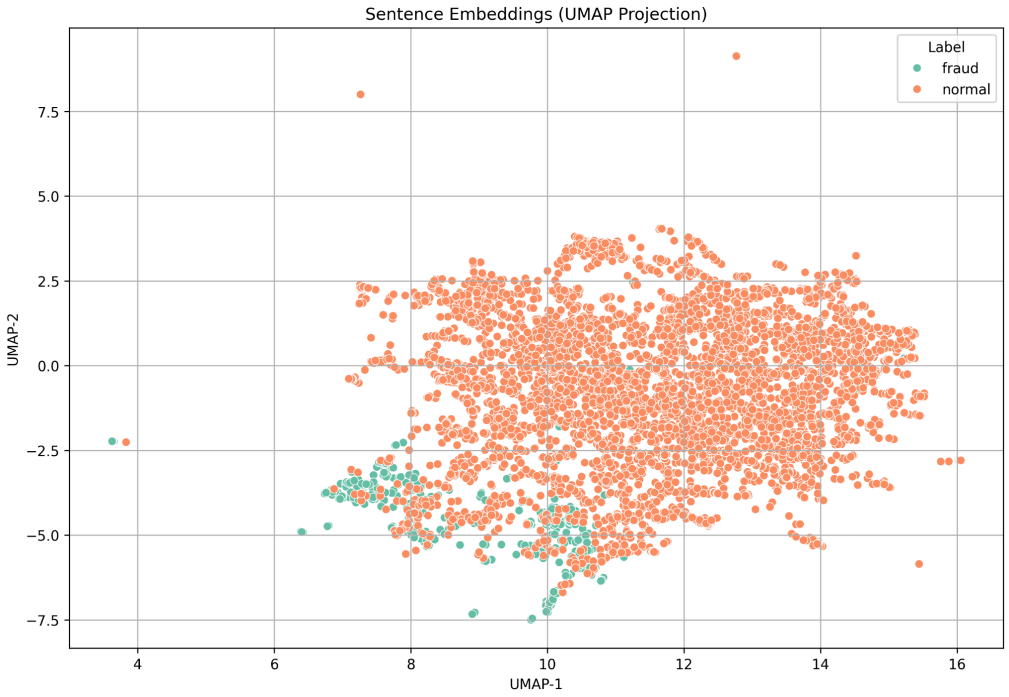




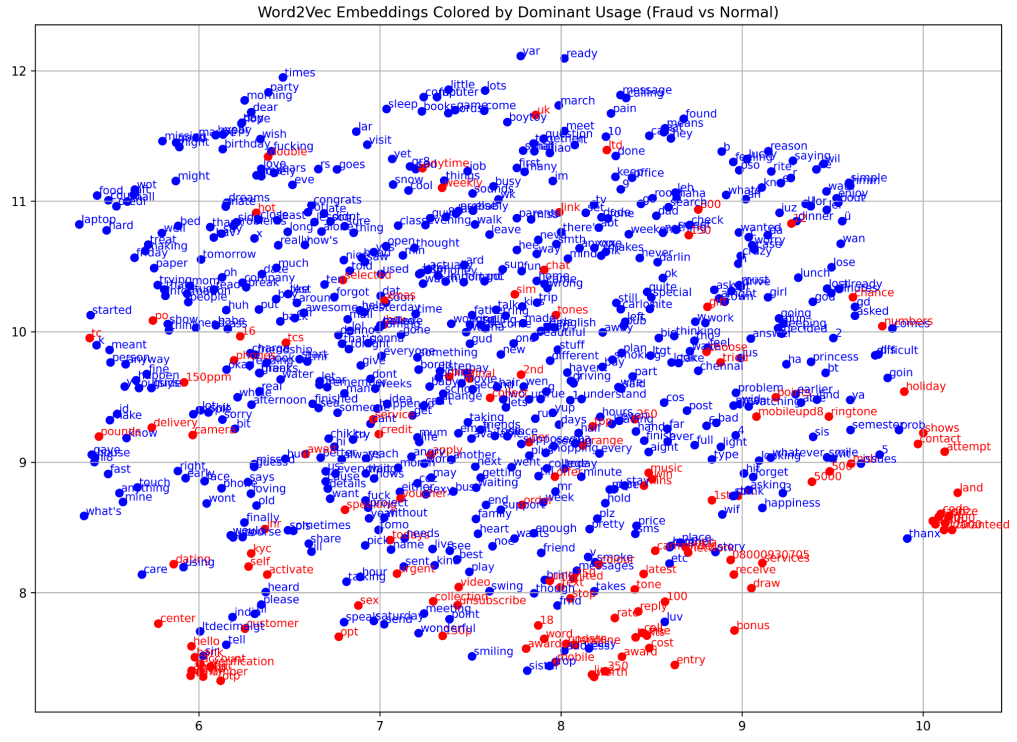


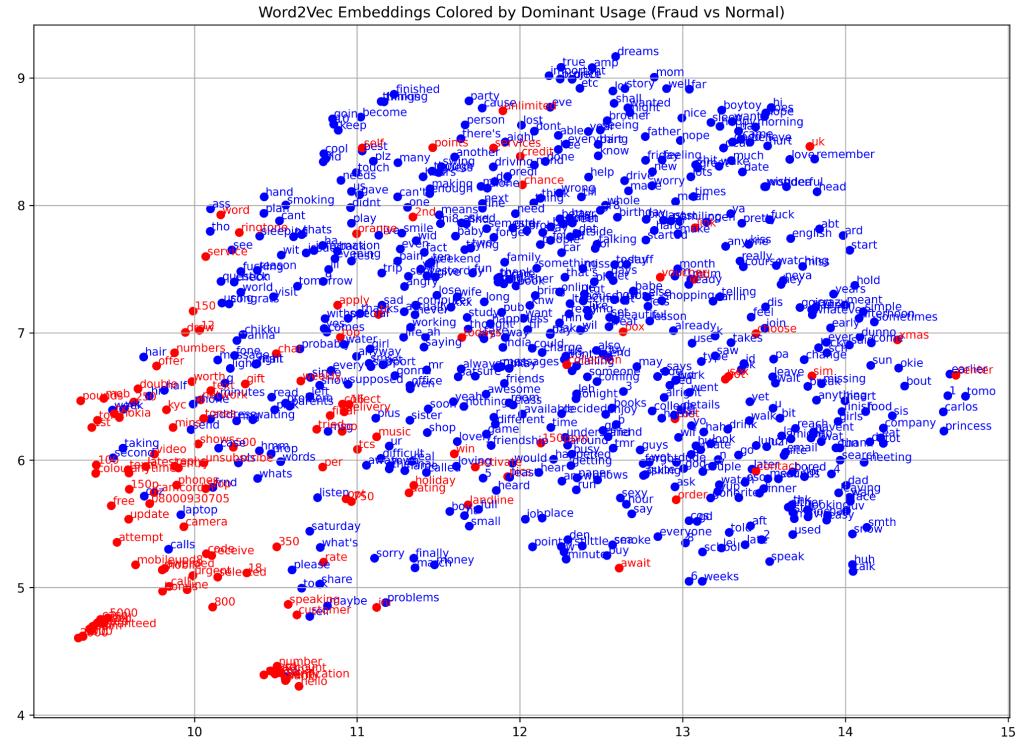


- Sentence embedding UMAP plots showing fraud and normal message separation, using KMeans Clustering

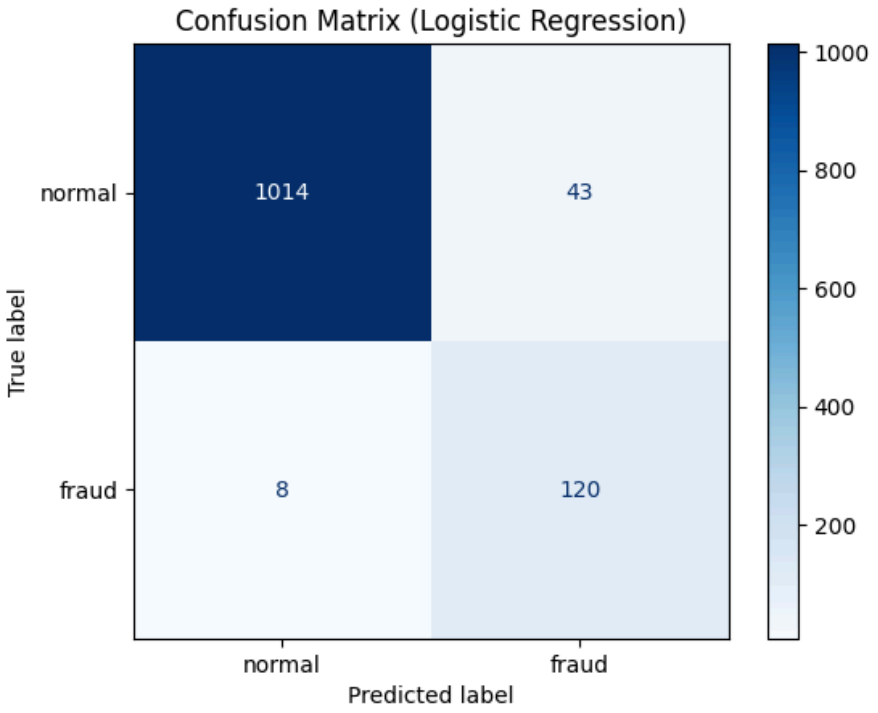


- Word2Vec embedding visualizations with fraud-dominant words highlighted





- Confusion matrix of classifier predictions on test data



- Classification report showing precision, recall, and F1-score

	precision	recall	f1-score	support
normal	0.99	0.96	0.98	1057
fraud	0.74	0.94	0.82	128
accuracy			0.96	1185
macro avg	0.86	0.95	0.90	1185
weighted avg	0.96	0.96	0.96	1185

Challenges Faced:

- Dataset formatting and missing values required multiple preprocessing iterations
- Learning DyNetX and Bokeh from scratch was time-
- Real-time rendering of large graphs was computationally heavy
- Community cluttering in visualizations required fine-tuning layout parameters
- Preprocessing challenges and overlapping vocabulary between fraud and normal messages while doing sentence embedding.

Evaluation Metrics/Accuracy:

- **Centrality Measures:** Degree, Betweenness, Closeness

  Top 5 nodes by degree centrality:-

  1. **have:** Centrality = 0.4506, Degree Count = 73
  2. **your**: Centrality = 0.3704, Degree Count = 60
  3. **hello**: Centrality = 0.3148, Degree Count = 51
  4. **that:** Centrality = 0.2407, Degree Count = 39
  5. **doing**: Centrality = 0.2407, Degree Count = 39

- **Graph Metrics:** Density, Clustering Coefficient, Average Path Length

- **Link Prediction and Classification (K Means) Metrics:** Precision, Recall, Accuracy (Temporal Holdout Validation)

Future Work / Conclusion:

- Explore enhanced community detection algorithms
- Simulate additional growth models like Barabási-Albert
- Apply advanced link prediction (e.g., Jaccard, GNN-based)
- Enhance dashboard UX and deploy as a web app
- Fine-tune the embedding model on fraud-specific language to improve sensitivity to scam patterns

Our project successfully demonstrates the application of temporal SNA for hoax call detection and offers a visual, data-driven tool for analyzing fraudulent messaging behavior over time and provides an extensible framework for dynamic fraud communication analysis.

Future work involves incorporating more sophisticated link prediction models (e.g., GNNs)xpanding to multi-modal fraud signals (text, call metadata), and enhancing real-time dashboard deployment for monitoring.